



Universidad Nacional Autónoma de México.

FACULTAD DE INGENIERÍA.

Proyecto Final

Reporte Técnico

Estructura y Programación de Computadoras

Profesora: Ing. Adara Mercado Martínez

-Integrantes:

- Carranza Gonzalez Alexis Aaron
 - Martínez Márquez Azael
 - Navarro Saldaña Ivan
 - Pérez González Natalia

Fecha de entrega: 11 de junio de 2026

1. Introducción

El presente reporte describe el diseño, la implementación y las pruebas de un sistema completo de traducción y construcción de programas para un subconjunto de la arquitectura IA-32. El sistema comprende un ensamblador de una pasada, un ensamblador de dos pasadas, generación de archivos objeto, manejo de símbolos y relocalaciones, y un mini enlazador (*linker*).

El objetivo principal del proyecto es comprender a profundidad el funcionamiento interno de los ensambladores, los formatos objeto, los enlazadores y la generación de código máquina, desarrollando todo desde cero en lenguaje C, sin hacer uso de herramientas automáticas como Yacc, Bison, Flex, ni ensambladores o enlazadores externos.

2. Objetivos

2.1 Objetivo General

Implementar un ensamblador modular para un subconjunto de IA-32 junto con un enlazador simplificado, desarrollando manualmente cada uno de sus componentes internos.

2.2 Objetivos Específicos

- Construir un lexer manual capaz de tokenizar código ensamblador IA-32.
- Construir un parser manual para el análisis sintáctico de instrucciones y directivas.
- Implementar tablas de símbolos para el manejo de etiquetas y referencias.
- Generar código máquina IA-32 correcto para el subconjunto de instrucciones soportadas.

- Implementar ensamblado de una y dos pasadas con soporte a referencias adelantadas.
- Diseñar e implementar un formato objeto con secciones, símbolos y tabla de relocalaciones.
- Construir un mini linker capaz de resolver símbolos externos y fusionar secciones.

3. Arquitectura del Sistema

El sistema se organiza en módulos independientes que cooperan a través de interfaces bien definidas en lenguaje C. Para demostrar la evolución del aprendizaje, el sistema soporta dos modalidades de ensamblado. A continuación se describe cada componente principal:

3.1 Lexer

El analizador léxico (`lexer.c`) es el componente base encargado de la lectura del archivo fuente y la tokenización. Su implementación manual ignora los comentarios (líneas iniciadas con `;`), descarta comas separadoras y clasifica los fragmentos de texto en *tokens* válidos. Es capaz de reconocer registros de 32 bits, constantes numéricas (decimales y hexadecimales), literales de texto (cadenas entre comillas) y validar identificadores para etiquetas, asegurando la consistencia léxica antes del análisis sintáctico.

3.2 Parser

El analizador sintáctico (`parser.c`) procesa los *tokens* para estructurarlos gramaticalmente. Se encarga del *parsing* de instrucciones, operandos y directivas de tamaño (como `DB`, `DW`, `DD`, `RESB`), así como de la validación sintáctica. Además, gestiona la variable de estado `current_section`, identificando transiciones entre las secciones `.TEXT`, `.DATA` y `.BSS` en mayúsculas para evitar desbordamientos, y construye la representación interna de los modos de direccionamiento complejos (evaluando corchetes, sumas y multiplicadores).

3.3 Tabla de Símbolos

Estructurada en `symtable.c`, actúa como el diccionario central del sistema. Maneja el registro dinámico de etiquetas, direcciones de memoria (`address`) e identificadores de sección (`section_id`). Incluye mecanismos de seguridad para la detección de

redefiniciones de etiquetas y permite la resolución de referencias cruzadas entre módulos mediante las banderas booleanas `is_global` e `is_extern`.

3.4 Ensamblador de Una Pasada

El ensamblador de una pasada implementado tiene como objetivo procesar el código fuente en un único recorrido, optimizando los tiempos de ensamblado. Para lograr esto, el sistema coordina la lectura, el análisis léxico y la emisión de código máquina en tiempo real, gestionando las referencias a etiquetas futuras mediante un sistema dinámico de parcheo (*fixups*) y culminando con la generación de un archivo objeto.

- **Lectura única y análisis del archivo:** El proceso inicia cargando la totalidad del archivo de código fuente en memoria utilizando la función `read_file()`. Posteriormente, se realiza el análisis léxico para generar la lista de *tokens*. El análisis sintáctico principal se ejecuta dentro de un ciclo `while` validado por la función `parser_has_more_tokens()`, el cual extrae y procesa una instrucción a la vez de forma *strictly* secuencial.
- **Generación de código máquina en tiempo real:** A medida que se lee cada instrucción, el contador de localidad (`loc_counter`) se actualiza continuamente, tomando como punto de partida la dirección base `0x08048000`. La función `encode_instruction()` evalúa los operandos de la instrucción extraída y emite los bytes correspondientes directamente a un búfer global de código máquina. Para asegurar la precisión geométrica de las direcciones de memoria, el sistema se apoya en `get_instruction_size()` para calcular e incrementar el `loc_counter` exacto de acuerdo a la arquitectura y tamaño de la instrucción procesada.
- **Manejo de referencias adelantadas mediante Fixups:** Al procesar instrucciones de salto o llamada (`JMP` o `CALL`) hacia una etiqueta en memoria, el sistema verifica primero si esta ya existe consultando a `symtable_find()`. Si la etiqueta aún no está definida, el ensamblador emite el *opcode* correspondiente seguido de ceros temporales mediante `emit_32bit(0)`, y registra esta referencia pendiente llamando a `add_fixup()`. La función `add_fixup()` almacena en una tabla el nombre de la etiqueta buscada, la dirección de la instrucción y el desplazamiento (*offset*) exacto dentro del

búfer de código máquina. Una vez que el ciclo de lectura encuentra la declaración real de esa etiqueta, esta se registra y se llama inmediatamente a `resolve_fixups()`. Esta función recorre la tabla de *fixups*, calcula el desplazamiento relativo real de 32 bits y sobrescribe los bytes ceros temporales directamente en los índices exactos del búfer original.

- **Gestión de la Tabla de Símbolos y Salida:** El archivo `symtable.c` mantiene un registro centralizado de los símbolos con una capacidad máxima definida por `MAX_SYMBOLS`. Si un símbolo se encuentra pendiente y se vuelve a encontrar en el flujo, se actualiza con `symtable_add()`, ajustando su dirección de memoria y cambiando su estatus a `defined = 1`. Finalmente, una vez que el ciclo concluye y todas las referencias posibles han sido resueltas, el programa vuelca el contenido del búfer global a un archivo físico mediante la llamada a `write_object_file("salida_1pasada.o")`.

3.5 Ensamblador de Dos Pasadas

Es la arquitectura principal y más avanzada del sistema. Resuelve el problema clásico de las referencias adelantadas y el manejo estructurado de múltiples secciones dividiendo la carga de trabajo en dos fases independientes:

- **Primera Pasada (Pass 1 - Mapeo de Memoria):** El sistema recorre el código fuente sin emitir código máquina. Su único objetivo es calcular los tamaños exactos de las instrucciones mediante `obtener_tamano_instruccion()` para incrementar los contadores `lc_text`, `lc_data` y `lc_bss` de forma independiente, construyendo una Tabla de Símbolos con posiciones de memoria absolutas y relativas perfectas.
- **Segunda Pasada (Pass 2 - Traducción):** Con todas las direcciones de las etiquetas resueltas y almacenadas, se invoca `parser_reset()`. El sistema recorre el flujo de tokens por segunda vez, utilizando la información recolectada para realizar la traducción definitiva de mnemónicos a código máquina y derivando los bytes al búfer de la sección correspondiente de manera ordenada.

3.6 Encoder IA-32

El módulo de codificación (`encoder.c`) es el motor de traducción matemática del sistema. Se encarga de la generación exacta de los *opcodes* binarios de la arquitectura IA-32, la construcción de los bytes **ModR/M** (necesarios para el direccionamiento de registros y accesos indirectos) y el byte **SIB** (Scale-Index-Base). Además, el *encoder* calcula e inyecta los valores de desplazamiento (*displacement*) y los operandos inmediatos, validando que las combinaciones de operandos sean permitidas por la arquitectura de Intel antes de emitir los bytes hacia los búferes de sección.

3.7 Formato Objeto y ObjWriter

Se diseñó e implementó un formato objeto binario personalizado estructurado para preservar la cohesión de los módulos compilados. El componente encargado de su persistencia es `objwriter.c`, el cual empaqueta los datos bajo la firma mágica corporativa **"UNAM"** siguiendo la siguiente distribución de bloques contiguos:

1. **[ObjHeader]**: Estructura de cabecera que incluye los metadatos globales, tamaño de la sección de código (`text_size`), tamaño de variables inicializadas (`data_size`), espacio reservado (`bss_size`), y los conteos finales de la tabla de símbolos y relocalizaciones.
2. **[Sección .TEXT]**: Bloque de memoria con los bytes puros de código máquina ejecutable generados en el Pass 2.
3. **[Sección .DATA]**: Bloque contiguo con los bytes asignados para constantes e inicializaciones de datos (procesando directivas DB, DW, DD y cadenas de texto OP_STRING).
4. **[Tabla de Símbolos]**: Arreglo estructurado de elementos `ObjSymbol` que exporta las etiquetas marcadas como globales (`is_global`) o externas (`is_extern`) para su posterior consumo.
5. **[Tabla de Relocalizaciones]**: Arreglo de elementos `ObjRelocation` que registra las posiciones relativas exactas dentro de `.TEXT` donde existen referencias a símbolos externos que el enlazador deberá parchar obligatoriamente.

3.8 Mini Linker

El subsistema de enlace (linker.c) lee uno o más archivos .o en formato UNAM y produce un ejecutable binario final unificado mediante un proceso secuencial de dos fases:

- **Fase 1 (Recolección de Símbolos):** Lee secuencialmente todos los archivos de entrada, concatena los búferes sumando los tamaños de las secciones y construye la tabla global de símbolos (global_syntable), ajustando de manera matemática las direcciones virtuales de cada módulo con base en el offset acumulado de los archivos precedentes.
- **Fase 2 (Parches de Relocalizaciones):** Re-inspecciona cada archivo objeto para procesar sus tablas de relocalización. Localiza el símbolo externo en la tabla global, calcula el offset relativo final bajo la ecuación $\text{Target} - (\text{Patch Address} + 4)$ y aplica los parches sobrescribiendo los 4 bytes correspondientes directamente en el buffer de salida final.

3.8.1 Registro de Desarrollo e Interacciones Iterativas

Para el diseño, desarrollo y robustecimiento de este subsistema de enlace, se ejecutaron las siguientes interacciones dirigidas con el modelo de IA:

- **Interacción 1 (Arquitectura del Formato Objeto):** Se solicitó adaptar el algoritmo de enlace base para que fuera 100% compatible con nuestra estructura nativa ObjHeader, la cual incrusta directamente los campos de tamaño contiguos en disco.
- **Interacción 2 (Soporte de Variables y Datos):** Se expandió la primera pasada del linker para que no solo fusionara código máquina de la sección de texto, sino que acoplara secuencialmente las secciones de datos (.data) y calculara los offsets lógicos de las variables en la memoria global.
- **Interacción 3 (Validación de Desbordamiento y Robustez):** Se implementó un bloque de control aritmético estricto dentro de la etapa de parcheo para blindar el cálculo del cálculo relativo contra desbordamientos de la arquitectura de destino.

3.8.2 Modificaciones Manuales y Decisiones de Ingeniería

- **Parchado en Little-Endian:** Se implementó y verificó manualmente el desempaque de la variable de dirección mediante máscaras de bits (& 0xFF)

y desplazamientos secuenciales (>> 8, >> 16, >> 24) para forzar la inyección de bytes en el orden Little-Endian nativo exigido por la arquitectura Intel x86.

- **Cumplimiento del Estándar C99 Extendido:** Se reestructuró la función `run_linker` para extraer y declarar todas las variables locales (`i`, `s`, `r`, `f`, `header`, etc.) en el inicio absoluto del bloque de código. Esto solucionó las restricciones estrictas de compilación del compilador GCC bajo entornos Linux/WSL de la facultad.

3.8.3 Errores Detectados y Soluciones Aplicadas

- **Error 1: Desfase por Instrucciones de Tamaño Variable (`location_counter`)**
 - *Detalle:* Inicialmente se asumía un avance estático de \$+4\$ bytes por instrucción de control de flujo. Al procesar código real con instrucciones de tamaño variable (como operaciones unarias o saltos condicionales cortos), los parches del linker destruían bytes contiguos de código máquina.
 - *Solución:* Se corrigió el pipeline matemático para evaluar dinámicamente el offset restando el tamaño exacto del espacio que ocupa el inmediato relativo de 32 bits, consolidando la fórmula formal: $\text{\textit{Target Absolute Address}} - (\text{\textit{Patch Address}} + 4)$.
- **Error 2: Violación de Segmento por Residuos en la RAM (*Garbage Patching*)**
 - *Detalle:* Los búferes globales de salida acumulaban basura electrónica de ejecuciones o llamadas pasadas si el sistema operativo no limpiaba las páginas de memoria asignadas al binario.
 - *Solución:* Se incrustó un bloque de sanitización forzada mediante llamadas explícitas a `memset(output_text_buffer, 0, sizeof(output_text_buffer))` al inicio de la función `run_linker`, garantizando parches limpios y reproducibles.
- **Error 3: Riesgo de Truncamiento Silencioso en Saltos de Gran Distancia**
 - *Detalle:* Al restar dos direcciones absolutas de memoria de 32 bits, la distancia calculada podía desbordar el rango de almacenamiento de un entero con signo al realizar conversiones implícitas de tipo.

- *Solución:* Se importó la librería estándar <limits.h> y se obligó al sistema a realizar la aritmética intermedia dentro de un contenedor rígido de 64 bits (int64_t large_offset). Se añadió una condicional de seguridad activa que aborta el enlazado (return 0) emitiendo un mensaje crítico en consola si el valor calculado excede los umbrales absolutos INT_MIN o INT_MAX.

4. Instrucciones y Directivas Soportadas

4.1 Registros

Se soportan los registros de propósito general de 32 bits: EAX, EBX, ECX, EDX, ESI, EDI, EBP y ESP.

4.2 Instrucciones

El ensamblador soporta el siguiente subconjunto de instrucciones IA-32:

- Transferencia: MOV, PUSH, POP, LEA.
- Aritméticas: ADD, SUB, INC, DEC, CMP, NEG, MUL, DIV.
- Lógicas: AND, OR, XOR, NOT.
- Saltos y control: JMP, JE, JNE, JG, JL, JGE, JLE, CALL, RET, NOP, INT.

4.3 Directivas

Se soportan las siguientes directivas: SECTION, GLOBAL, EXTERN, DB, DW, DD, ORG, EQU, RESB, RESW, RESD.

5. Modos de Direccionamiento

El ensamblador implementa los siguientes modos de direccionamiento:

- Inmediato: MOV EAX, 10
- Registro a registro: MOV EAX, EBX
- Memoria directa: MOV EAX, [1000]
- Base + desplazamiento: MOV EAX, [EBP+4]
- Base + índice: MOV EAX, [EBX+ECX]
- Base + índice escalado: MOV EAX, [EBX+ECX*4]
- Base + índice escalado + desplazamiento: MOV EAX, [EBX+ECX*4+8]

6. Soporte SIB (Scale Index Base)

El proyecto implementa soporte parcial para el byte SIB. Se soportan las escalas 1, 2, 4 y 8. Para una instrucción como MOV EAX, [EBX+ECX*4+8], el codificador construye internamente la secuencia hexadecimal **8B 44 8B 08**, desglosada de la siguiente manera:

- **Opcode de la instrucción (8B):** Corresponde a la instrucción principal MOV r32, r/m32.
- **Byte ModR/M (44):**
 - **Mod (01):** Indica la presencia de un desplazamiento numérico de 8 bits.
 - **Reg (000):** Codifica el registro destino (EAX).
 - **R/M (100):** Este valor especial exige la evaluación obligatoria del byte SIB continuo.
- **Byte SIB (8B):**
 - **Scale (10):** Factor de escala multiplicador de $\times 4$.
 - **Index (001):** Codifica el registro índice (ECX).
 - **Base (011):** Codifica el registro base (EBX).
- **Displacement (08):** Como el campo Mod se fijó en 01, se inyecta un byte con el valor directo del desplazamiento (0x08).
- **Immediate (si aplica):** Para esta variante de la instrucción no se requiere un operando inmediato extra.

7. Casos de Prueba

Esta sección describe los casos de prueba diseñados para validar el correcto funcionamiento del sistema. Cada caso fue ejecutado sobre el ensamblador de una pasada y el de dos pasadas de forma independiente, verificando que ambos produzcan el mismo código objeto. Se indica el archivo de entrada, el comportamiento esperado y el resultado obtenido.

7.1 MOV Inmediato y Operaciones Básicas (test_basic.asm)

Ensamblador de una pasada

Este caso verifica que el ensamblador de una pasada sea capaz de traducir correctamente instrucciones de transferencia con operando inmediato, transferencia registro a registro, operaciones aritméticas (ADD, SUB, INC, DEC), operaciones lógicas (AND, OR, XOR) e interrupción de software (INT 0x80). El programa define además secciones .data y .bss con las directivas DB, DW, DD y RESB, lo que permite verificar que el ensamblador gestiona correctamente múltiples secciones en una sola pasada.

Dado que todos los símbolos del programa son locales y están definidos antes de ser usados, el ensamblador de una pasada no necesita emitir fixups: el código máquina puede generarse de forma completa e inmediata durante la lectura del archivo. El archivo objeto resultante (test_basic.o, 23 bytes) contiene el código de la sección .text codificado en el formato objeto propio del proyecto.

Resultado: el ensamblador generó el archivo objeto sin errores. El código producido es correcto para todas las instrucciones del caso.

Ensamblador de dos pasadas

En la primera pasada se construye la tabla de símbolos con las etiquetas _start y las variables var_byte, var_word, var_dword y buffer, calculando sus offsets dentro de cada sección. En la segunda pasada se genera el código máquina definitivo y se produce el archivo objeto. Al no existir referencias adelantadas ni símbolos externos, ambas pasadas concluyen sin pendientes en la tabla de fixups.

Resultado: idéntico al del ensamblador de una pasada. Ambas implementaciones producen el mismo archivo objeto de 9 bytes.

7.2 Saltos y Referencias Adelantadas (test_jumps.asm)

Ensamblador de una pasada

Este caso tiene como objetivo principal validar el mecanismo de referencias adelantadas. El programa contiene dos saltos condicionales, JE y JG, que apuntan a las etiquetas salto_adelante y fin_programa respectivamente. Ambas etiquetas se encuentran definidas más adelante en el archivo, por lo que en el momento en que el ensamblador procesa los saltos su dirección de destino aun es desconocida.

El ensamblador de una pasada maneja esta situación emitiendo el opcode del salto con un desplazamiento provisional de cero y registrando un fixup en la tabla de pendientes. Una vez que el ensamblador alcanza la definicion de la etiqueta correspondiente, calcula el desplazamiento relativo correcto (relativo al siguiente byte tras el salto) y aplica el parche sobre el buffer de codigo ya generado. El archivo objeto resultante (test_jumps.o, 86 bytes) refleja los desplazamientos resueltos.

Resultado: los fixups fueron resueltos correctamente. Los bytes de desplazamiento de JE y JG en el archivo objeto corresponden a los offsets calculados respecto a sus destinos.

Ensamblador de dos pasadas

En la primera pasada el ensamblador recorre el archivo completo sin emitir codigo, unicamente construyendo la tabla de simbolos. Al finalizar esta pasada, las direcciones de salto_adelante y fin_programa son conocidas. En la segunda pasada se genera el codigo maquina con los desplazamientos ya resueltos desde el inicio, sin necesidad de fixups.

Resultado: identico al del ensamblador de una pasada. El archivo objeto de 28 bytes es equivalente byte a byte en ambas implementaciones.

7.3 Directivas ORG y EQU (test_org.asm)

Ensamblador de una pasada

Este caso verifica el soporte a las directivas ORG y EQU. La directiva ORG 0x1000 indica al ensamblador que el contador de programa debe iniciarse en la direccion 0x1000, lo que afecta el calculo de todas las direcciones del programa. Las constantes STDIN y STDOUT se definen mediante EQU y se usan como operandos inmediatos en instrucciones MOV. El ensamblador de una pasada procesa EQU al vuelo, registrando las constantes en la tabla de simbolos para sustituirlas cuando aparecen como operandos.

Resultado: el archivo objeto generado (test_org.o, 15 bytes) refleja el desplazamiento de base indicado por ORG y los valores correctos de las constantes en los operandos inmediatos.

Ensamblador de dos pasadas

En la primera pasada se registran STDIN, STDOUT y SYSCALL como simbolos de tipo constante en la tabla. ORG ajusta el contador de programa desde el inicio de la primera pasada, de modo que todos los offsets calculados son relativos a 0x1000. La segunda pasada emite el codigo con las constantes ya sustituidas.

Resultado: identico al de una pasada. Archivo objeto de 15 bytes equivalente.

7.4 Multiples Modulos y Relocaciones (test_linker_mod1.asm + test_linker_mod2.asm)

Ensamblador de una pasada

Este es el caso de prueba mas completo del proyecto. El modulo principal (test_linker_mod1.asm) cubre la totalidad del subconjunto de instrucciones soportadas: MOV en todos sus modos, instrucciones aritmeticas, logicas, todos los saltos condicionales e incondicional, CALL a un simbolo externo, PUSH, POP, LEA, MUL, DIV, NOP, RET e INT. Ademas hace uso de las directivas EQU, SECTION, GLOBAL, EXTERN, DB, DW, DD, RESB, RESW y RESD.

La instruccion CALL funcion_externa genera una entrada en la tabla de relocaciones del archivo objeto, ya que funcion_externa esta declarada como EXTERN y su direccion sera resuelta por el linker. El ensamblador emite el opcode de CALL con un desplazamiento provisional de cero y registra la relocacion con el nombre del simbolo y el offset dentro de la seccion .text donde debe aplicarse el parche.

El modulo secundario (test_linker_mod2.asm) define y exporta el simbolo funcion_externa mediante GLOBAL, con un cuerpo minimo de ADD EAX, 50 seguido de RET. Su archivo objeto (test_linker_mod2.o, 15 bytes) contiene la seccion .text con esas dos instrucciones y una entrada en la tabla de simbolos exportados.

Resultado: ambos modulos fueron ensamblados correctamente. test_linker_mod1.o (111 bytes) contiene la tabla de relocaciones con la entrada para funcion_externa. test_linker_mod2.o (15 bytes) exporta el símbolo correctamente.

Ensamblador de dos pasadas

En la primera pasada del módulo principal se construye la tabla de símbolos con todas las etiquetas locales (igual, diferente, mayor, menor, mayor_igual, menor_igual, fin, mensaje, buffer) y se registra `funcion_externa` como símbolo externo pendiente. En la segunda pasada se genera el código con todos los saltos resueltos localmente y se emite la entrada de relocación para `funcion_externa`.

Resultado: idéntico al de una pasada. Los archivos objeto producidos son equivalentes en contenido.

7.5 Mini Linker: Resolución de Símbolos Externos

Una vez ensamblados los dos módulos anteriores, el mini linker procesa `test_linker_mod1.o` y `test_linker_mod2.o`. El proceso consiste en: leer los encabezados y tablas de símbolos de ambos archivos objeto; fusionar las secciones `.text` de ambos módulos en un único segmento de código; construir una tabla global de símbolos con los símbolos exportados por cada módulo; recorrer la tabla de relocalaciones del módulo principal y aplicar el parche sobre el offset indicado, sustituyendo el desplazamiento provisional de `CALL` por el desplazamiento relativo correcto hacia `funcion_externa` en el segmento fusionado; y finalmente escribir el binario resultante.

El binario final generado (`ejecutable_final.bin`, 16 bytes) contiene el código de ambos módulos fusionado, con la instrucción `CALL` correctamente parcheada para apuntar a `funcion_externa`.

Resultado: el linker resolvió el símbolo externo sin errores y generó el binario final con la relocación aplicada correctamente.

7.6 Direccionamiento SIB (`test_sib.asm`)

Ensamblador de una pasada

Este caso valida la codificación del byte SIB (Scale Index Base) en sus distintas variantes. El programa ejercita los modos de memoria directa (`MOV EAX, [1000]`), base más desplazamiento (`MOV EDX, [EBP+4]`), base más índice sin escala (`MOV ESI, [EBX+ECX]`) y la forma completa con escala y desplazamiento (`MOV EAX, [EBX+ECX*4+8]`).

Para la instruccion MOV EAX, [EBX+ECX*4+8] el encoder construye internamente: el opcode 0x8B, el byte ModRM con mod=10 (desplazamiento de 8 bits), reg=000 (EAX) y rm=100 (indica presencia de SIB); el byte SIB con escala=10 (factor 4), index=001 (ECX) y base=011 (EBX); el desplazamiento de 8 bits con valor 0x08; sin byte inmediato. El archivo objeto resultante (test_sib.o, 9 bytes) contiene la secuencia de bytes correspondiente a estas instrucciones.

Resultado: el encoder genero correctamente el byte SIB para todas las variantes de direccionamiento probadas, incluyendo la forma completa con escala 4 y desplazamiento.

Ensamblador de dos pasadas

Al no existir referencias adelantadas ni simbolos externos en este modulo, la primera pasada unicamente registra la etiqueta _start y calcula los offsets de cada instruccion. La segunda pasada invoca al encoder para cada instruccion y produce el mismo archivo objeto que el ensamblador de una pasada.

Resultado: identico al de una pasada. Archivo objeto de 9 bytes equivalente byte a byte.

7.7 Manejo de Error: Simbolo Externo No Resuelto (error_fantasma.asm)

Este caso verifica el comportamiento del sistema ante un error semantico: el modulo declara como EXTERN el simbolo funcion_que_no_existe y lo invoca con CALL, pero no existe ningun otro modulo que lo defina y exporte. El ensamblador produce el archivo objeto (error_fantasma.o, 39 bytes) con la entrada de relocacion pendiente, ya que su responsabilidad es unicamente ensamblar el modulo y registrar los simbolos externos.

El error se detecta en la fase del linker: al intentar resolver la tabla de relocalaciones, el linker no encuentra funcion_que_no_existe en ninguna tabla de simbolos exportados y emite un mensaje de error indicando el nombre del simbolo no resuelto y el modulo que lo referencia. El proceso de enlace se aborta y no se genera binario de salida.

Resultado: el sistema detecto y reporto correctamente el simbolo externo no resuelto. Este caso valida tanto la correcta generacion de entradas de relocacion por parte del ensamblador como la deteccion de errores de enlace en el linker.

8. Manejo de Errores

8.1 Errores de uso (invocación incorrecta)

Estos errores ocurren antes de procesar cualquier archivo, cuando el programa se invoca sin los argumentos necesarios.

Ensamblador de 2 pasadas sin argumentos:

Uso: ./assembler_2pass <archivo.asm>

Uso Linker: ./assembler_2pass --link <archivo1.o> <archivo2.o> ... -o <ejecutable>

Se imprime en stderr y el programa termina con código de salida 1. Muestra ambas formas de uso del binario: modo ensamblador y modo linker.

Ensamblador de 1 pasada sin argumentos:

Uso: ./assembler_1pass <archivo_ensamblador>

8.2 Errores de entrada/salida

Relacionados con la lectura de archivos fuente o la escritura de archivos de salida.

Archivo fuente no encontrado o no legible:

Emitido por read_file() en main_2pass.c cuando fopen() retorna NULL. El programa termina con código 1.

Versión de 1 pasada (Memoria insuficiente al leer el archivo (1 pasada)):

Ocurre si malloc() falla al reservar el buffer para el contenido de la fuente. En la práctica sólo aparecería en sistemas con memoria muy limitada.

No se puede crear el archivo objeto de salida:

Emitido por `generate_obj_file()` en `objwriter.c` cuando no se puede abrir el archivo de salida para escritura (por ejemplo, por permisos insuficientes en el directorio).

8.3 Errores léxico

El lexer genera un token de tipo `TOKEN_ERROR` cuando encuentra un carácter que no pertenece a ninguna categoría válida del lenguaje ensamblador. Caracteres como `@`, `#`, `$`, `%`, `?` o cualquier símbolo no definido producen este token.

El lexer no interrumpe la tokenización: registra el carácter problemático como `TOKEN_ERROR` con su lexema y número de línea, y continúa procesando el resto del archivo. El token de error se almacena en la lista junto con todos los demás.

El parser, al encontrar un `TOKEN_ERROR` dentro de un operando, no puede clasificarlo como ningún tipo de operando válido. La función `parse_operand()` ejecuta `advance()` y retorna un operando de tipo `OP_NONE`. Como consecuencia, la instrucción se construye con un operando vacío, y el encoder la rechaza en la fase 2

El error se reporta en `stdout` indicando la instrucción afectada. El ensamblador no se detiene: continúa procesando el resto de instrucciones, lo que significa que un carácter ilegal en una línea no impide la generación del archivo `.o`, aunque ese archivo contendrá código incorrecto para esa instrucción

8.4 Errores semánticos

8.4.1 Etiqueta definida más de una vez

Detectado durante la **pasada 1** del ensamblador de 2 pasadas, cuando `symtable_find()` devuelve un símbolo ya marcado como `defined=1` al intentar añadir una etiqueta con el mismo nombre, este es el único error que provoca una **terminación inmediata con `exit(1)`**. La ejecución no continúa porque una etiqueta duplicada hace que la tabla de símbolos sea ambigua y cualquier código generado a partir de ese punto sería incorrecto. El número de línea no se incluye en el mensaje, aunque la detección ocurre en el orden de parseo de la pasada 1.

8.4.2 Etiqueta no definida en saltos (2 pasadas)

Cuando en la pasada 2 el encoder encuentra un JMP o instrucción de salto condicional cuyo operando es una etiqueta que no está en la tabla de símbolos

[Encoder Error] Etiqueta 'ETIQUETA_INEXISTENTE' no definida en este archivo.

El mensaje incluye el nombre exacto de la etiqueta. El encoder omite los bytes de esa instrucción y continúa. El comportamiento es no fatal: el archivo .o se genera igualmente, pero el salto queda sin codificar.

8.4.3 Instrucción o combinación de operandos no soportada

El encoder de 2 pasadas cubre un subconjunto del ISA IA-32. Cuando recibe una instrucción o una combinación específica de operandos para la que no tiene una regla de codificación, al final de `encode_instruction()` cae en el caso por defecto:

[Encoder Error] Instruccion o combinación no soportada.

Este mensaje aparece en stdout, identifica que hubo un problema pero no especifica la instrucción ni el número de línea en el propio mensaje de error.

8.4.4 Tabla de fixups llena (1 pasada)

El ensamblador de 1 pasada reserva una tabla de fixups con capacidad máxima de 256 entradas. Si hay más de 256 saltos hacia adelante a etiquetas no definidas aún en el momento del salto, la tabla se desborda

Error: Tabla de fixups llena

El mensaje se emite en stderr por cada intento de añadir un fixup más allá del límite. El ensamblador continúa ejecutando pero los saltos excedentes quedarán con offset 0x00000000 sin corregir, produciendo un archivo objeto con código de salto incorrecto para esas instrucciones.

8.5 Errores del Linker

8.5.1 Archivo objeto no encontrado

Cuando `fopen()` falla al intentar abrir alguno de los archivos .o de entrada. El linker termina con retorno 0 (sin ejecutable generado).

8.5.2 Firma mágica inválida

El linker verifica los primeros 4 bytes de cada archivo con `strncmp(header.magic, "UNAM", 4)`. Si el archivo no tiene la firma correcta, el linker rechaza ese archivo y termina. Esto protege contra intentar enlazar archivos ELF, archivos de texto u otros binarios no relacionados.

[Linker Error] Firma magica 'UNAM' no encontrada en archivo.o

8.5.3 Símbolo externo no resuelto

Ocurre durante la fase de parche de relocalizaciones cuando una entrada de la tabla de relocalizaciones referencia un símbolo que no aparece en ninguno de los módulos de entrada (o que aparece pero con `is_defined=0`). El linker termina sin generar el ejecutable.

Este es el error semántico más importante del sistema de enlazado: indica que un módulo declara una dependencia (EXTERN) que ningún otro módulo satisface (GLOBAL).

[Linker Error] Simbolo externo no resuelto: 'FUNCION_QUE_NO_EXISTE'

8.5.4 Overflow de distancia de salto

El linker calcula el offset relativo usando `int64_t` antes de hacer el cast a `int32_t`, y verifica que el valor esté dentro del rango `[INT_MIN, INT_MAX]`. Si dos módulos están tan separados en el espacio de direcciones que la distancia entre ellos no cabe en 32 bits con signo, el linker aborta con este mensaje y muestra el valor exacto del offset que causó el desbordamiento. En la práctica, con un buffer máximo de 64 KB definido por `MAX_OUTPUT_SIZE`, este error no se alcanza en condiciones normales, pero la verificación existe como salvaguarda.

[Fatal Linker Error] Distancia de salto demasiado grande para 'simbolo'.

8.5.5 Error de escritura del ejecutable

Cuando `fopen()` falla al crear el archivo de salida, generalmente por falta de permisos en el directorio destino.

[Linker Error] Error fatal de escritura al crear 'ejecutable.bin'

9. Organización del Equipo

El proyecto fue desarrollado por cuatro integrantes, con la siguiente distribución de responsabilidades:

Integrante 1 — Lexer y Tokens

Ensamblador de una pasada

El trabajo desarrollado para este módulo consistió en la construcción del analizador léxico (*Lexer*), el cual actúa como la primera etapa de procesamiento del ensamblador. Su función principal es transformar el código fuente en texto plano hacia una secuencia estructurada de *tokens*.

- Se implementó la estructura *Lexer* para mantener el estado del análisis, controlando el cursor de lectura y el número de línea actual para facilitar el rastreo de errores.
- La función `tokenize()` recorre el archivo fuente carácter por carácter, ignorando eficientemente saltos de línea, espacios en blanco y comentarios iniciados con el carácter `;`.
- Se desarrolló la función `classify_lexeme()` para categorizar cadenas de texto específicas, distinguiendo con precisión entre registros de la arquitectura IA-32, mnemónicos de instrucciones analizadas y directivas de control del ensamblador (`SECTION`, `GLOBAL`, etc.).
- El módulo extrae operandos alfanuméricos y numéricos mediante bucles de validación con las librerías estándar `isalpha()` e `isdigit()`, convirtiendo automáticamente el texto a mayúsculas para mantener consistencia en las siguientes etapas de evaluación.

- La gestión de memoria para el arreglo final de *tokens* se diseñó de forma dinámica, utilizando malloc y realloc para expandir la capacidad del búfer automáticamente cuando la cantidad de elementos detectados supera la capacidad inicial reservada.

Ensamblador de dos pasadas

El trabajo desarrollado para este módulo consistió en la construcción del analizador léxico (*Lexer*), el cual actúa como la primera etapa de procesamiento, transformando el código fuente crudo en una secuencia estructurada de *tokens*.

- Se definió la enumeración TokenType para representar exhaustivamente todos los elementos del lenguaje, incorporando tokens específicos para agrupaciones (TOKEN_LBRACKET, TOKEN_RBRACKET), operadores para cálculo de direcciones (TOKEN_PLUS, TOKEN_STAR) y literales de texto (TOKEN_STRING).
- La función tokenize() procesa el texto ignorando espacios en blanco, contabilizando los saltos de línea para el rastreo de errores y descartando eficientemente los comentarios iniciados con el carácter ;.
- Se implementó una gestión dinámica de memoria que inicializa el búfer con una capacidad de 100 *tokens*, utilizando realloc para duplicar este espacio automáticamente cuando el arreglo está por llenarse.
- La subrutina classify_lexeme() clasifica las cadenas alfanuméricas con alta precisión, distinguiendo entre registros, instrucciones avanzadas de la arquitectura (incluyendo IMUL, IDIV), y directivas críticas para el control del Linker y las variables (SECTION, GLOBAL, EXTERN, DB, RESB, etc.).
- El módulo incluye una lógica de extracción especializada que detecta comillas dobles ("), iterando sobre la cadena interna para empaquetar el texto completo como un único TOKEN_STRING, facilitando su posterior escritura en la sección .DATA.

Integrante 2 — Parser y Representación Interna

Ensamblador de una pasada

El desarrollo de este módulo se centró en el análisis sintáctico (*Parser*), encargado de tomar el flujo de *tokens* generado por el Lexer y estructurarlo en una representación interna lógica para el ensamblador.

- Se definió la estructura *Instruction* para encapsular el mnemónico, la línea de origen y hasta dos operandos, junto con la estructura *Operand* que clasifica el tipo de direccionamiento utilizado.
- La función `parse_next_instruction()` fue diseñada para consumir los *tokens* secuencialmente, identificando la presencia de etiquetas (verificando el `TOKEN_COLON`) y asignando los operandos correspondientes a la instrucción.
- Se implementó una lógica de clasificación de operandos en `parse_operand()`, capaz de distinguir entre registros (`OP_REGISTER`), valores inmediatos (`OP_IMMEDIATE`) y referencias directas a memoria (`OP_DIRECT_MEMORY`).
- El módulo incluye un análisis avanzado para el direccionamiento indirecto (`OP_INDIRECT_MEMORY`), procesando correctamente la sintaxis de corchetes y calculando los registros base, índice, escala y desplazamiento mediante la detección de `TOKEN_PLUS` y `TOKEN_STAR`.
- La gestión del flujo se controla mediante un índice global y la función auxiliar `advance()`, asegurando que el análisis se detenga correctamente al alcanzar el `TOKEN_EOF`.

Ensamblador de dos pasadas

El desarrollo se centró en adaptar el análisis sintáctico para soportar el manejo de múltiples secciones de memoria (`.TEXT`, `.DATA`, `.BSS`) mediante la actualización de la variable global `current_section`.

- Se integró el procesamiento de la directiva `EQU`, permitiendo definir constantes que se registran directamente con su valor inmediato antes de evaluar otras instrucciones.
- La función `parse_next_instruction()` se encarga de aislar etiquetas, mnemónicos y sincronizar el número de línea actual (`sync_line`) para evitar desfases al procesar saltos de línea y operandos.

- Se robusteció `parse_operand()` para soportar direccionamiento indirecto complejo de la arquitectura IA-32.
- El sistema itera sobre *tokens* como `TOKEN_PLUS` y `TOKEN_STAR` para extraer registros base, índices matemáticos y la escala multiplicadora directamente desde los corchetes.

Integrante 3 — Backend del Ensamblador

Ensamblador de una pasada

La responsabilidad de este componente fue orquestar el flujo principal de ejecución del ensamblador de una pasada, gestionando la memoria, la tabla de símbolos y la resolución de referencias adelantadas.

- Se construyó el ciclo principal de ensamblado en `main()`, el cual procesa instrucción por instrucción manteniendo un contador de localidad (`loc_counter`) que inicia en la dirección virtual `0x08048000` de la arquitectura IA-32.
- Se implementó un sistema de parcheo (*Fixups*) para resolver el problema de las referencias futuras en un diseño de una sola lectura.
- Cuando el código hace referencia a una etiqueta no definida, la función `add_fixup()` guarda el nombre de la etiqueta buscada, la dirección actual de la instrucción y el índice (*offset*) exacto dentro del búfer donde deberán insertarse los bytes faltantes.
- Una vez que la etiqueta es declarada posteriormente en el código, el sistema invoca `resolve_fixups()` para calcular la distancia relativa de 32 bits y sobrescribe los ceros temporales directamente en los índices guardados del búfer.
- El módulo integra la lectura inicial del código fuente mediante la función de reserva dinámica `read_file()`.

Ensamblador de dos pasadas

Se implementó la arquitectura central del ensamblador en `main_2pass.c`, dividiendo la ejecución en dos fases claramente definidas: un mapeo de memoria inicial (Pass 1) y una traducción a lenguaje máquina (Pass 2).

- Durante la primera pasada, el sistema utiliza la función `obtener_tamano_instruccion()` para incrementar independientemente los contadores de localidad `lc_text`, `lc_data` y `lc_bss` de acuerdo a la sección de memoria activa en ese instante.
- El módulo gestiona la tabla de símbolos validando duplicados mediante `syntable_find()` y diferenciando el registro entre símbolos definidos internamente, símbolos externos mediante la directiva `EXTERN`, y símbolos exportables con `GLOBAL`.
- En la segunda pasada, se reinicia el estado del analizador llamando a `parser_reset()` para volver a leer el archivo de inicio a fin.
- Una vez reiniciado, se procede a llamar a la rutina de codificación `encode_instruction()` enviando los diferentes búferes para generar los bytes binarios de manera definitiva.

Integrante 4 — Encoder IA-32

Ensamblador de una pasada

Este módulo se dedicó a la traducción de las estructuras internas generadas por el *Parser* a los códigos de máquina binarios específicos de la arquitectura IA-32 y en el empaquetado del código de máquina en un formato de archivo binario estructurado para preservar la información generada por el ensamblador. *(Nota: Al ser el ensamblador de una pasada, no se requirió la fase de Linker).*

- Se desarrolló la función base `encode_instruction()` que evalúa el mnemónico y la combinación exacta de operandos para emitir el *opcode* hexadecimal correcto hacia el `machine_code_buffer`.
- El módulo incluye rutinas a nivel de bits como `build_modrm()` y `build_sib()` para construir dinámicamente los bytes de direccionamiento requeridos por la arquitectura.
- Se implementó la función de mapeo `get_register_code()`, la cual convierte las cadenas de texto de los registros (ej. "EAX", "ECX") a sus valores numéricos estandarizados del procesador (0 a 7).
- Para garantizar que el contador de localidad del Backend se mantenga preciso, se creó la función `get_instruction_size()`, la cual calcula por

adelantado la cantidad exacta de bytes que ocupará una instrucción antes de ser emitida al búfer.

- Las escrituras a memoria se manejan de forma segura comprobando que no se exceda el límite de 8192 bytes definido para el arreglo global mediante la función `emit_byte()`.
- Se definió la cabecera personalizada `ObjHeader` que incluye la firma ("magic bytes") "UNAM", el tamaño de la sección de texto y un contador de símbolos.
- Se implementó la función `write_object_file()` que recibe el nombre del archivo de salida (ej. "salida_1pasada.o") y se encarga de abrir un flujo de escritura binaria segura usando `fopen` con el modo `wb`.
- El módulo vuelca de manera contigua primero la estructura `ObjHeader` y posteriormente el volcado completo del `machine_code_buffer` utilizando la función estándar `fwrite()`.

Ensamblador de dos pasadas

Para esta versión del ensamblador, el módulo de codificación fue reestructurado para soportar la emisión de código en múltiples secciones y preparar las instrucciones para el proceso de enlazado y también se construyó el motor de generación de archivos objeto y un enlazador dinámico capaz de empaquetar y unificar los códigos de máquina para su ejecución.

- El módulo se rediseñó para emitir código máquina de manera diferenciada según la sección activa (`.TEXT` o `.DATA`), utilizando funciones especializadas como `emit_byte_to_section()` y `emit_32bit_to_section()` para direccionar los bytes al búfer correcto.
- Se implementó una resolución inteligente de operandos en `encode_instruction()`: si el sistema detecta una referencia a memoria directa (`OP_DIRECT_MEMORY`) cuya etiqueta ya está definida en la tabla de símbolos, el codificador transforma dinámicamente el operando para utilizar su dirección absoluta como un valor inmediato (`OP_IMMEDIATE`).
- El procesamiento de direccionamiento indirecto fue ampliado para soportar la construcción dinámica del byte `SIB` (Scale-Index-Base) mediante `build_sib()`,

evaluando factores de escala (2, 4 u 8) y ajustando el ModR/M según los desplazamientos requeridos.

- Se integró la comunicación directa con el enlazador dinámico: al procesar una instrucción CALL hacia una etiqueta que no está definida localmente, el codificador emite el *opcode* seguido de ceros temporales y registra la ubicación exacta llamando a `retable_add()` para su futura resolución.
- Se añadió el procesamiento de directivas de inicialización de memoria (DB, DW, DD), traduciendo tanto valores numéricos como iterando sobre arreglos de caracteres (OP_STRING) para inyectar su representación hexadecimal contigua en el búfer de la sección de datos.
- La función `generate_obj_file()` estructura una cabecera `ObjHeader` que contiene la firma mágica "UNAM", junto con los tamaños exactos y contadores requeridos.
- El sistema empaqueta secuencialmente los búferes binarios de texto y datos, seguidos de la tabla de símbolos exportables (`ObjSymbol`) y la tabla de reubicaciones pendientes (`ObjRelocation`).
- Se desarrolló un enlazador (*Linker*) en la función `run_linker()` que consolida múltiples archivos objeto, sumando sus tamaños globales y ajustando las direcciones absolutas de las variables para evitar colisiones.
- El enlazador recorre las tablas de reubicación buscando referencias en `global_symtable`, calcula las distancias relativas exactas de 32 bits, y aplica los parches sobrescribiendo los índices del código máquina.
- Se incluyeron mecanismos de seguridad para detectar desbordamientos de la arquitectura, abortando automáticamente el proceso si la distancia de salto supera los límites de un número entero de 32 bits (`INT_MIN` o `INT_MAX`).

10. Conclusiones

Cuando empezamos el proyecto, la verdad no imaginamos todo lo que implicaba hacer un ensamblador. Sonaba fácil: tomar instrucciones y convertirlas a código máquina. Pero en cuanto comenzamos a programarlo nos dimos cuenta de que había muchos problemas que nunca habíamos considerado.

Uno de los primeros obstáculos fueron las referencias hacia adelante. Cuando una instrucción hacía un salto a una etiqueta que todavía no había sido definida, nuestro programa no sabía qué dirección colocar. Al principio simplemente no contemplamos ese caso y varios programas dejaban de funcionar. Después implementamos fixups, guardando temporalmente un valor y corrigiéndolo más adelante cuando ya conocíamos la dirección real. Más adelante también probamos el enfoque de dos pasadas, que resultó mucho más ordenado porque primero identifica todas las etiquetas y después genera el código. Gracias a eso entendimos que ensamblar no es únicamente traducir instrucciones una por una.

Otro tema que nos costó fue el byte ModRM. Al revisar la documentación de Intel parecía que todo estaba lleno de excepciones y reglas difíciles de entender. Muchas veces nos preguntábamos por qué ciertas instrucciones usaban determinados opcodes o por qué algunas variantes requerían bytes adicionales. Sin embargo, cuando implementamos la función para construir el ModRM y comenzamos a generar los bytes manualmente, fue más sencillo. Esto nos ayudó a entender mucho mejor cómo se representan realmente las instrucciones dentro del procesador.

El linker también fue una de las partes más interesantes. Al construir uno propio entendimos el trabajo que realiza al unir distintos módulos, reorganizar direcciones y resolver referencias externas.

Respecto al formato UNAM, decidimos mantenerlo lo más sencillo posible. Tener una estructura clara y sencilla nos facilitó mucho las pruebas y la depuración. Cuando investigamos formatos reales como ELF nos dimos cuenta de que son mucho más complejos debido a la gran cantidad de características que soportan. Aunque nuestro formato es mucho más simple, cumplió perfectamente su propósito y nos ayudó a comprender cómo se organiza la información dentro de un archivo objeto.

En general, lo más importante del proyecto fue que nos permitió entender todo el proceso que ocurre entre escribir un programa y obtener un ejecutable. Muchas cosas que antes parecían automáticas ahora tienen sentido porque tuvimos que implementarlas nosotros mismos.

11. Criterios de Evaluación

Rubro	Porcentaje
Ensamblador de 1 pasada	20%
Ensamblador de 2 pasadas	20%
Formato objeto	15%
Mini linker	20%
Manejo de errores	10%
Pruebas	5%
Reporte técnico	5%
Bitácora IA	5%